

PROGRAMACIÓN 2 · C++

Enumeradores

Un tipo propio para darle nombre a los valores

¿Qué significa ese 0?

- Representamos colores con un `int` y un «acuerdo secreto»: 0 rojo, 1 verde, 2 amarillo.
- Para la máquina solo hay números. Para el que lee el código... «0» no dice nada.

```
int colorManzana = 0; // 0 = rojo (acuerdo secreto)

if (colorManzana == 0) {
    cout << "Manzana roja" << endl;
}
```

Idea clave: estos son los «números mágicos»: valores que nadie entiende de qué se tratan.

Darle nombre a los números

- Guardamos cada valor en una constante `const int` con nombre.
- El código deja de usar el «0» crudo: ahora se lee solo.

```
const int ROJO    = 0;
const int VERDE   = 1;
const int AMARILLO = 2;

int colorManzana = ROJO;

if (colorManzana == ROJO) {
    cout << "Manzana roja" << endl;
}
```

Mejor, pero todavía: `colorManzana` sigue siendo un `int`.

Nada restringe los valores

- El `int` acepta **cualquier** número.
- Quien sea puede escribir `8...` y el compilador no se queja.

```
int colorManzana = 8; // compila sin problema... ¡pero no es un color!
```

Idea clave: queremos que el compilador conozca cuáles son los valores válidos.

bool ya hace algo parecido

- `true` y `false`: dos **valores nombrados**, y no hay otros.
- `bool` es un **tipo propio**: el compilador lo distingue de `int`, `char`, etc.

```
bool encendido = true;
bool apagado   = false;

if (encendido) {
    cout << "Encendido" << endl;
}
```

Mentor: ¿y si pudiéramos crear nuestros propios tipos, con los valores que elijamos y con nombre? Eso existe. Se llaman **enums**.

enum: valores con nombre

Un **enumerador** es un tipo definido por el usuario que asigna *nombres* a valores enteros. Sin números mágicos:

```
enum Dia {  
    LUNES,      // 0  
    MARTES,     // 1  
    MIERCOLES,  // 2  
    JUEVES,     // 3  
    VIERNES,    // 4  
    SABADO,     // 5  
    DOMINGO     // 6  
};  
  
Dia hoy = LUNES;
```

Detalle: los valores se asignan solos, empezando en 0 y siguiendo la secuencia.

No mezclar tipos

Cada enum es **un tipo diferente**. El compilador no permite combinar valores de enums distintos:

```
enum Color { NEGRO, ROJO, AZUL };  
enum Mascota { GATO, PERRO, CERDO };  
  
Mascota miMascota{ ROJO }; // ERROR: ROJO no es un Mascota  
Color camisa{ CERDO }; // ERROR: CERDO no es un Color
```

Nombres: el tipo con mayúscula inicial (`Dia` , `Color`); los valores en minúscula.

Tus valores, tus reglas

- Podés asignar el valor que quieras a cada nombre.
- Donde no asignás, la secuencia continúa.

```
enum Mes {  
    ENERO    = 1,  
    FEBRERO  = 2,  
    // ... hasta DICIEMBRE = 12  
    SIN_MES  = 0  
};  
  
enum Nivel {  
    BAJO = 10,  
    MEDIO,      // 11 (sigue la secuencia)  
    ALTO,        // 12  
    CRITICO = 100 // salto intencional  
};
```

Ámbito propio y tipo seguro

enum clásico: sin control

```
enum Color    { ROJO, AZUL };  
enum Semaforo { ROJO, ... };  
// ERROR: nombre duplicado  
int x = ROJO; // se convierte solo
```

enum class: a salvo

```
enum class Color    { ROJO, AZUL };  
enum class Semaforo { ROJO, ... };  
  
Color c = Color::ROJO; // prefijo obligatorio  
int x = Color::ROJO;   // ERROR: no convierte
```

Idea clave: cada valor vive en su propio ámbito y no se convierte a `int` por accidente.

La firma delata qué entra

Al declarar un enum creamos un **tipo nuevo**: lo podemos usar como parámetro y el compilador rechaza valores fuera de lugar.

Con int: cualquier cosa entra

```
void configurar(int estado);
```

```
configurar(42); // compila
```

```
configurar(-3); // compila
```

Con enum class: solo lo válido

```
void configurar(Estado estado);
```

```
configurar(Estado::ACTIVO); // OK
```

```
configurar(42); // ERROR
```

```
ej_error.cpp:9:18: error: cannot convert 'int' to 'Estado' for argument '1' to 'void configurar(Estado)'
```

switch + enum class = código que se lee solo

```
enum class Dia { LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO };

void saludo(Dia dia) {
    switch (dia) {
        case Dia::SABADO:
        case Dia::DOMINGO:
            cout << "Hoy es día de descanso!" << endl;
            break;
        case Dia::VIERNES:
            cout << "Último esfuerzo, ya queda poco!" << endl;
            break;
        default:
            cout << "A estudiar C++!" << endl;
            break;
    }
}
```

Salida de saludo(Dia::MARTES); saludo(Dia::VIERNES); saludo(Dia::SABADO):

```
A estudiar C++!
Último esfuerzo, ya queda poco!
Hoy es día de descanso!
```

Reglas de oro

- Usá `enum class` siempre que puedas: es la versión moderna y segura.
- Los enums terminan con los **números mágicos**: el código se lee solo.
- Usalos como **parámetros** para rechazar valores inválidos en compilación.
- No hay conversión implícita a `int`: si la necesitás, pedila explícitamente.

«El código se lee mucho más de lo que se escribe.»